

— NATURAL LANGUAGE PROCESSING

LECTURE 3

Feature Extraction

Bag of Words (BoW), TF-IDF & Cosine Similarity

Learning Outcomes

By the end of this lecture, students will be able to:

- 01** Explain the purpose of Feature Extraction.
- 02** Build a Bag of Words model.
- 03** Create a Document–Term Matrix.
- 04** Calculate TF and IDF.
- 05** Compute TF–IDF scores.
- 06** Measure document similarity using Cosine Similarity.
- 07** Implement these techniques using Python and Scikit–learn.

Review of Previous Lecture

NLP Pipeline

Key Question

Can Machine Learning understand text directly?

Answer: No. ML algorithms require numerical input.



What is Feature Extraction?

DEFINITION

Feature Extraction is the process of converting textual data into numerical representations that machine learning algorithms can understand.

Without Feature Extraction:

- ✗ No Text Classification
- ✗ No Sentiment Analysis
- ✗ No Search Engine
- ✗ No Recommendation System

RUNNING EXAMPLE

Use these documents throughout the lecture:

D1 "I love Natural Language Processing"

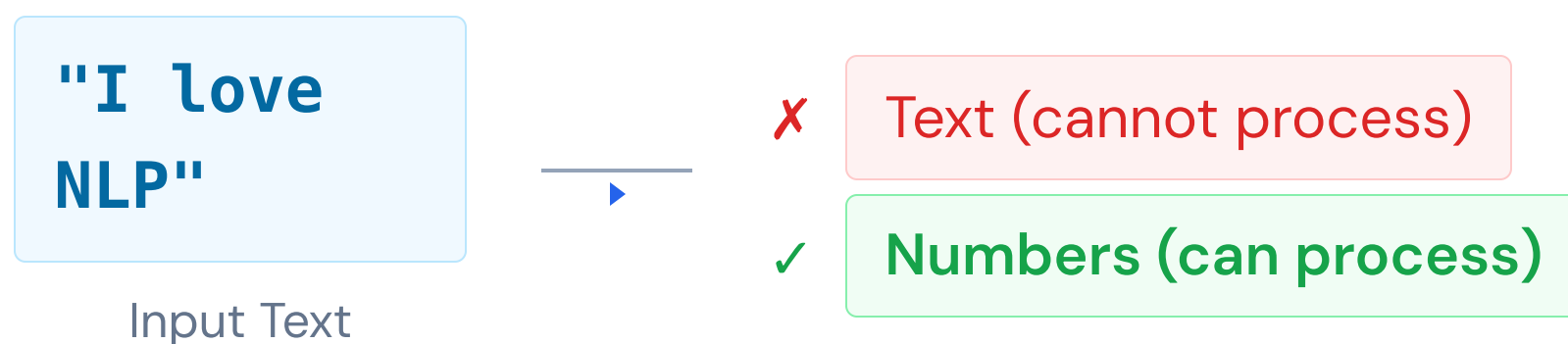
D2 "Natural Language Processing is interesting"

D3 "I love Machine Learning"

These same documents will be used in all examples throughout this lecture.

Why Feature Extraction?

Computers process **numbers**, not words.



Feature Extraction transforms text into vectors that machine learning models can process.

Bag of Words (BoW)

DEFINITION

Bag of Words is a feature extraction technique that represents each document by counting the frequency of each word.

Characteristics

- Ignores grammar.
- Ignores word order.
- Counts only word occurrences.

EXAMPLE

"I love NLP"



{ I: 1, love: 1, NLP: 1 }

Each unique word becomes a **dimension** in the document vector, with its count as the value.

Building the Vocabulary

DOCUMENTS

D1: I love NLP

D2: I love AI

D3: AI loves NLP

Every unique word appears **only once** in the vocabulary, regardless of how many times it appears in the corpus.

VOCABULARY

Index	Word
1	I
2	love
3	loves
4	NLP
5	AI

Document-Term Matrix

Document	I	love	loves	NLP	AI
D1	1	1	0	1	0
D2	1	1	0	0	1
D3	0	0	1	1	1

Rows

Documents

Columns

Vocabulary

Cells

Word Frequency

Manual Example

SHARED VOCABULARY

- Machine
- Learning
- is
- amazing

This is how text becomes numerical data.

Document A

"Machine Learning is amazing"

Vector: [1, 1, 1, 1]

Document B

"Machine Learning"

Vector: [1, 1, 0, 0]

Word	Doc A	Doc B
Machine	1	1
Learning	1	1
is	1	0
amazing	1	0

Advantages & Limitations of BoW

+ Advantages

- ✓ Easy to understand.
- ✓ Easy to implement.
- ✓ Fast.
- ✓ Good baseline model.

- Limitations

- ✗ Ignores word order.
- ✗ Ignores context.
- ✗ Ignores semantics.
- ✗ Produces sparse matrices.

BoW is a strong baseline — but its limitations motivate more advanced techniques like TF-IDF.

Term Frequency (TF)

DEFINITION

Term Frequency (TF) measures how often a word appears in a document.

A word that appears more frequently is generally considered more important *within that document*.

Key Idea

- **High TF** → Word is important in the document.
- **Low TF** → Word is less important.

FORMULA

$$\text{TF}(t, d) = \frac{\text{Number of occurrences of } t}{\text{Total number of words in document}}$$

Where **t** = term, **d** = document

Quick Example

If "AI" appears 3 times in a 10-word document:

$$\text{TF}(\text{"AI"}, d) = 3 / 10 = 0.30$$

TF Example

DOCUMENT

"AI AI Machine Learning AI"

Total words: 5

Observation

The document mainly discusses AI, because it has the highest TF value.

TF CALCULATION

Word	Count	TF
AI	3	$3/5 = 0.60$
Machine	1	$1/5 = 0.20$
Learning	1	$1/5 = 0.20$

$TF("AI", d) = 3 / 5 = 0.60$ — highest in this document

Why TF is Not Enough?

Consider three documents:

D1: The cat is sleeping.

D2: The dog is running.

D3: The bird is flying.

Problem

TF measures frequency, **not importance across the document collection.**

HIGH-TF WORDS IN EVERY DOCUMENT

"the"

appears in all 3 docs

"is"

appears in all 3 docs

⚠ High TF, but very little useful information!

This limitation motivates:

Inverse Document Frequency (IDF)

IDF penalizes words that appear in many documents — giving more weight to rare, informative words.

Inverse Document Frequency (IDF)

DEFINITION

Inverse Document Frequency (IDF) measures how rare or unique a word is across an entire collection of documents.

- Words appearing in **many documents** receive a **low IDF**.
- Words appearing in **few documents** receive a **high IDF**.

FORMULA

$$\text{IDF}(t) = \frac{\log (N)}{\log (df(t))}$$

N = Total number of documents

df(t) = Number of documents containing the term

Intuition

The logarithm prevents extreme values. A word in **every** document has $\text{IDF} = \log(1) = 0$ — it carries no discriminative information.

IDF Example

Collection size: **4** documents

Interpretation

- Common words receive **small IDF values**.
- Rare words receive **large IDF values** because they carry more information.

IDF VALUES

Word	Appears in	IDF
AI	4 documents	Low
Python	1 document	High
Neural	1 document	High

$$\text{IDF}(\text{"AI"}) = \log(4/4) = 0 \mid \text{IDF}(\text{"Python"}) = \log(4/1) = 1.386$$

TF vs IDF

TF — Term Frequency	IDF — Inverse Document Frequency
Measures frequency inside one document	Measures rarity across all documents
Local importance	Global importance
High TF → Frequent word	High IDF → Rare word
Cannot distinguish common words	Reduces the weight of common words

TF answers:

"How important is a word in this document?"

IDF answers:

"How unique is this word across the collection?"

Conclusion:

TF and IDF **complement each other** — combining them gives TF-IDF.

TF-IDF

Term Frequency – Inverse Document Frequency

DEFINITION

TF-IDF is a feature extraction technique that combines TF and IDF to measure the importance of a word in a document relative to a collection of documents.

Key Idea — A word scores high if:

- It appears **frequently** in one document. (High TF)
- It appears **rarely** in the entire document collection. (High IDF)

FORMULA

$$\text{TF-IDF}(t, d) = \text{TF}(t, d) \times \text{IDF}(t)$$

TF(t, d)	×	IDF(t)
local importance		global rarity

Why TF-IDF?

- ✗ TF alone cannot distinguish common words.
- ✗ IDF alone ignores how often a word appears in a document.
- ✓ **TF-IDF combines the strengths of both.**

TF-IDF Worked Example

Suppose we are given:

Word	TF	IDF
AI	0.60	0.22
Python	0.20	1.38

Observation

Although **Python** appears fewer times, it receives a **higher TF-IDF score** because it is much rarer across the document collection.

CALCULATE TF-IDF

AI

TF × IDF

0.60 × 0.22 = 0.132

Python

TF × IDF

Higher Score

0.20 × 1.38 = 0.276

Word	TF-IDF Score	Rank
Python	0.276	#1
AI	0.132	#2

TF-IDF Matrix

After calculating TF-IDF for every word, we build a **TF-IDF Matrix**.

Document	AI	Machine	Learning	Python
D1	0.13	0.07	0.07	0.00
D2	0.00	0.08	0.08	0.15
D3	0.07	0.00	0.00	0.18

Rows

Represent documents

Columns

Represent terms

Values

Word importance
(not count)

Unlike BoW, values are **weighted**, not simple frequencies.

Bag of Words vs TF-IDF

Bag of Words	TF-IDF
Counts word frequency	Assigns importance weights
Treats all words equally	Reduces common words
Integer values	Decimal weights
Simpler representation	More informative representation

EXAMPLE — THE WORD "THE"

Bag of Words:

High frequency → Looks important.

TF-IDF:

Low weight → Recognized as a common word.

Conclusion

TF-IDF produces a **better numerical representation** for most NLP tasks because it weighs words by their true importance.

Applications of TF-IDF

TF-IDF is widely used in many NLP applications:

Key Takeaway

TF-IDF remains one of the most widely used feature extraction techniques because it is **simple, efficient, and highly effective.**

01 Information Retrieval

02 Search Engines

03 Document Ranking

04 Spam Detection

05 Text Classification

06 Keyword Extraction

07 Document Clustering

08 Recommendation Systems

Cosine Similarity

DEFINITION

After converting text into vectors using BoW or TF-IDF, we need a way to compare documents.

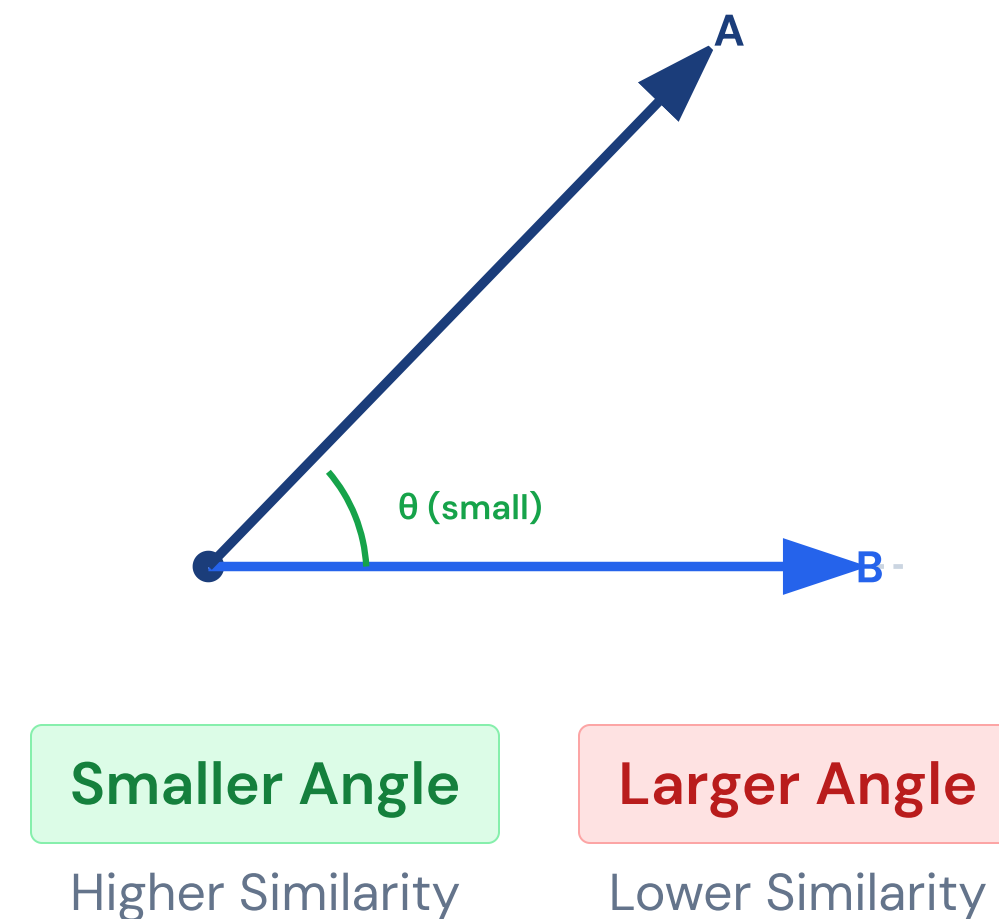
Cosine Similarity measures the similarity between two document vectors by calculating the **cosine of the angle** between them.

Key Idea

- **Small angle** → High similarity.
- **Large angle** → Low similarity.

Unlike Euclidean Distance, Cosine Similarity focuses on the **direction** of the vectors rather than their length.

VISUALIZATION



Cosine Similarity Formula

The similarity between two vectors **A** and **B** is:

COSINE SIMILARITY =

$$\frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \times \|\mathbf{B}\|}$$

$\mathbf{A} \cdot \mathbf{B}$ = Dot Product of vectors A and B

$\|\mathbf{A}\|$ = Magnitude (length) of vector A

$\|\mathbf{B}\|$ = Magnitude (length) of vector B

OUTPUT RANGE

Value	Meaning
1	Identical vectors (Maximum similarity)
0	No similarity
-1	Opposite direction (rare in text analysis)

In Practice

For text documents, values typically fall between 0 and 1, since word counts are non-negative.

Worked Example

Suppose we have two document vectors:

$$\mathbf{A} = [1, 1, 1]$$

$$\mathbf{B} = [1, 1, 0]$$

Interpretation

Since **0.816** is close to 1, the two documents are **highly similar**.

Step 1 Dot Product

$$(1 \times 1) + (1 \times 1) + (1 \times 0) = 1 + 1 + 0 = 2$$

Step 2 Magnitude

Vector A:

$$\sqrt{(1^2 + 1^2 + 1^2)} = \sqrt{3} \approx 1.732$$

Vector B:

$$\sqrt{(1^2 + 1^2 + 0^2)} = \sqrt{2} \approx 1.414$$

Step 3 Cosine Similarity

$$\frac{2}{\sqrt{3} \times \sqrt{2}} = \frac{2}{2.449} \approx 0.816$$

Why Cosine Similarity?

Suppose:

A = [1, 1]

B = [10, 10]

Although **B** is much larger than **A**, both vectors point in the **same direction**.

Key Insight

Document length should not determine similarity. Cosine Similarity compares the **content**, not the size of the document.

✗ Euclidean Distance

Considers A and B **far apart** because their magnitudes differ significantly.

$$\text{distance}(A, B) = \sqrt{(10-1)^2 + (10-1)^2} \approx 12.73$$

✓ Cosine Similarity

Considers A and B **very similar** because their direction is the same.

$$\text{cosine}(A, B) = 1.0 \quad (\text{identical direction})$$

Two short docs and two long docs on the same topic should be equally similar — Cosine Similarity achieves this.

Applications of Cosine Similarity

Cosine Similarity is a **foundational technique** in NLP, powering many real-world systems.

Why it Works

Language data is high-dimensional and sparse. Cosine Similarity handles this efficiently by focusing on **direction**, not magnitude.

01 Document Similarity

Finding similar articles, papers, or news items.

02 Recommendation Systems

Suggesting similar products or content based on user behaviour.

03 Plagiarism Detection

Identifying documents with very high similarity scores.

04 Information Retrieval

Matching user queries to relevant documents in search engines.

05 Question Answering

Finding the most relevant passage that answers a user question.

Python Lab — scikit-learn

We will use **scikit-learn** to implement everything we have learned — BoW, TF-IDF, and Cosine Similarity — in Python.

LAB OBJECTIVES

- 01 Apply CountVectorizer for Bag of Words.
- 02 Apply TfidfVectorizer for TF-IDF.
- 03 Apply cosine_similarity for similarity.

REQUIRED LIBRARY

```
from sklearn.feature_extraction.text
import CountVectorizer, TfidfVectorizer
from sklearn.metrics.pairwise
import cosine_similarity
```

Over the next three slides:

- CountVectorizer — Bag of Words
- TfidfVectorizer — TF-IDF
- cosine_similarity — document comparison

CountVectorizer

Implements **Bag of Words**
— converts a collection of
text documents to a matrix
of token counts.

OUTPUT

X: Sparse matrix (documents
× vocabulary)

get_feature_names_out():
Vocabulary list

```
bow_example.py

from sklearn.feature_extraction.text import CountVectorizer

# Sample documents
docs = [
    "I love Natural Language Processing",
    "Natural Language Processing is interesting",
    "I love Machine Learning"
]

# Create vectorizer and fit
vectorizer = CountVectorizer()
X = vectorizer.fit_transform(docs)

# View vocabulary and matrix
```

TfidfVectorizer

Implements **TF-IDF** — converts text to a matrix of TF-IDF features, weighting each word by its importance.

VS COUNTVECTORIZER

✗ CountVectorizer treats all words equally.

✓ TfidfVectorizer weights words by true importance.

The API is **identical** to CountVectorizer — just swap the class name.

```
tfidf_example.py

from sklearn.feature_extraction.text import TfidfVectorizer

# Same documents
docs = [
    "I love Natural Language Processing",
    "Natural Language Processing is interesting",
    "I love Machine Learning"
]

# TF-IDF vectorizer
tfidf = TfidfVectorizer()
X = tfidf.fit_transform(docs)

# View vocabulary and TF-IDF matrix
```


cosine_similarity

cosine_similarity from scikit-learn computes pairwise cosine similarity between document vectors in one call.

READING THE OUTPUT

- Cell `[i][j]` = similarity of doc `i` and doc `j`.
- Diagonal is always **1.0** (a document is identical to itself).
- Matrix is **symmetric**: `[i][j] == [j][i]`.

D1 and D2 share **0.63** similarity — both discuss NLP. D1 and D3 share

```
cosine_example.py
```

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
```

```
docs = [
    "I love Natural Language Processing",
    "Natural Language Processing is interesting",
    "I love Machine Learning"
]
```

```
# Build TF-IDF matrix
```

```
tfidf = TfidfVectorizer()
```

```
X = tfidf.fit_transform(docs)
```

```
# Compute pairwise cosine similarity
```

Chapter Summary

In this lecture we covered three fundamental techniques for converting text into numerical representations.

Why These Techniques Matter

Every NLP system — from search engines to chatbots — relies on some form of feature extraction before any learning can occur.

01 Bag of Words (BoW)

Converts text to word count vectors. Simple but ignores word order and context.

02 TF-IDF

Weights words by local frequency and global rarity. More informative than raw counts.

03 Cosine Similarity

Measures the angle between two document vectors. Length-independent document comparison.

Text → BoW / TF-IDF → Vectors → Cosine Similarity → Score

Key Takeaways

These five points capture everything you must remember from Lecture 3.

Mastering these concepts gives you the foundation to understand all modern NLP systems — from classical search to transformer-based models.

01

Feature Extraction converts text into numerical vectors so machine learning models can process it.

02

Bag of Words counts word occurrences but ignores word order and context.

03

TF-IDF improves on BoW by weighting words — high if they appear often in a document and rarely across the collection.

04

Cosine Similarity measures the angle between two document vectors, independent of document length.

05

scikit-learn provides `CountVectorizer`, `TfidfVectorizer`, and `cosine_similarity` to implement all three techniques in Python.

Quiz

Test your understanding of Feature Extraction, TF-IDF, and Cosine Similarity.

Instructions

- Answer individually first.
- Discuss answers in pairs.
- Review as a class.

Q1 What is Feature Extraction and why is it needed in NLP?

Q2 What is the main difference between Bag of Words and TF-IDF?

Q3 Why does a word that appears in every document receive an IDF score of 0?

Q4 Given vectors $A = [1, 0, 1]$ and $B = [1, 1, 0]$, calculate their Cosine Similarity.

Q5 Name two real-world applications that use Cosine Similarity.

DEPARTMENT OF ARTIFICIAL INTELLIGENCE · THIRD YEAR

Thank You

Lecture 3 — Feature Extraction

Bag of Words

TF-IDF

Cosine Similarity

NEXT LECTURE

Word Embeddings — Word2Vec & GloVe